

ĐẠI HỌC BÁCH KHOA
ĐẠI HỌC QUỐC GIA TP HCM
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



HỌC MÁY
(C03117)

Báo cáo tiến độ

Lần thứ tư - ngày 26/04/2026

Giảng viên hướng dẫn: TS. Trương Vĩnh Lân
Nhóm thực hiện: Nhóm 10

Họ và tên	MSSV	Ghi chú
Trần Gia Hân	2052464	Nhóm trưởng <i>han.tran2202@hcmut.edu.vn</i>
Lê Minh Mẫn	2312040	
Nguyễn Thành Trình	2313640	
Ngô Quang Tân	2313052	
Lương Mạnh Tiến	2213459	

Mục lục

1	Logistic Regression (mô hình hồi quy logistic – Baseline)	3
1.1	Lý do chọn	3
1.2	Cấu hình huấn luyện	3
1.3	Kết quả trên tập Validation	3
1.4	Kết quả trên tập Test	3
1.5	Nhận xét	4
2	MLP – Multi-Layer Perceptron (mạng nơ-ron nhiều lớp)	4
2.1	Kiến trúc mô hình	4
2.2	Cấu hình huấn luyện	5
2.3	Các kỹ thuật được áp dụng	5
2.3.1	BatchNorm1d (chuẩn hóa theo lô – Batch Normalization)	5
2.3.2	Dropout(p=0.5)	5
2.3.3	Early Stopping (dừng huấn luyện sớm, patience = 15)	5
2.3.4	ReduceLROnPlateau	5
2.4	Diễn biến quá trình huấn luyện	6
2.5	Kết quả trên tập Test	6
2.6	Nhận xét	7
3	So sánh tổng hợp: Logistic Regression và MLP	7
3.1	Bảng so sánh	7
3.2	Phân tích từng chỉ số	7
3.2.1	Accuracy (độ chính xác tổng thể)	7
3.2.2	Precision (độ chính xác dự đoán dương – lớp >50K)	7
3.2.3	Recall (độ phủ lớp dương – lớp >50K)	7
3.2.4	F1-Score (trung bình điều hòa của Precision và Recall – lớp >50K)	8
3.2.5	ROC-AUC (diện tích dưới đường cong ROC – đo khả năng phân biệt hai lớp)	8
3.3	Kết luận	8
4	Đánh giá chung	8
4.1	Ưu điểm đã đạt được	8
4.2	Hạn chế và điểm cần cải thiện	9
5	Hướng phát triển tiếp theo	9
5.1	Thử nghiệm thêm kiến trúc MLP	9
5.2	Hyperparameter Tuning có hệ thống (tinh chỉnh siêu tham số)	10
5.3	Xử lý mất cân bằng lớp – so sánh thêm phương pháp	10
5.4	Hoàn thiện notebook và cấu trúc dự án	10



5.5	Báo cáo cuối kỳ	11
-----	---------------------------	----

1 Logistic Regression (mô hình hồi quy logistic – Baseline)

1.1 Lý do chọn

Logistic Regression (*hồi quy logistic*) được chọn làm **baseline** (mốc so sánh nền) vì ba lý do chính:

- Đơn giản, huấn luyện nhanh và dễ giải thích kết quả.
- Hoạt động hiệu quả với dữ liệu dạng bảng đã được chuẩn hóa và mã hóa.
- Kết quả của nó là ngưỡng tối thiểu mà MLP cần vượt qua để chứng minh lợi ích thực sự của học sâu trên bài toán này.

1.2 Cấu hình huấn luyện

Table 1: Cấu hình Logistic Regression

Tham số	Giá trị
Solver	lbfgs
max_iter	1.000
random_state	42
Dữ liệu train	Tập train sau SMOTE (47.618 mẫu, cân bằng 50/50)

1.3 Kết quả trên tập Validation

Table 2: Kết quả Logistic Regression – Validation Set

Nhãn	Precision	Recall	F1-score
$\leq 50K$	0.94	0.79	0.86
$> 50K$	0.57	0.85	0.68

Accuracy: 0.8041 **ROC-AUC:** 0.9056

1.4 Kết quả trên tập Test

Table 3: Kết quả Logistic Regression – Test Set

Nhãn	Precision	Recall	F1-score	Support
$\leq 50K$	0.94	0.79	0.86	6.803
$> 50K$	0.57	0.84	0.68	2.242

Accuracy: 0.8048 **Precision** (>50K): 0.5719 **Recall** (>50K): 0.8443

F1-Score (>50K): 0.6819 **ROC-AUC:** 0.9038

1.5 Nhận xét

Kết quả giữa tập val (0.8041) và tập test (0.8048) rất nhất quán, xác nhận mô hình tổng quát hóa tốt và không bị *overfitting* (quá khớp trên dữ liệu huấn luyện). Chỉ số ROC-AUC = 0.9038 cho thấy khả năng phân biệt hai lớp thu nhập ở mức khá tốt tính trên toàn bộ ngưỡng quyết định. Đây là ngưỡng tham chiếu nền tảng để đánh giá mô hình MLP ở phần tiếp theo.

2 MLP – Multi-Layer Perceptron (mạng nơ-ron nhiều lớp)

2.1 Kiến trúc mô hình

Mô hình được xây dựng bằng **PyTorch** với kiến trúc ba lớp ẩn **[256, 128, 64]**. Mỗi lớp ẩn đều theo cùng một cấu trúc: **Linear** → **BatchNorm1d** → **ReLU** → **Dropout**.

Table 4: Kiến trúc MLP [256, 128, 64]

Lớp (Layer)	Chi tiết
Input Layer	87 đặc trưng đầu vào (sau OHE + StandardScaler)
Hidden Layer 1	Linear (87→256) → BatchNorm1d (256) → ReLU → Dropout (p=0.5)
Hidden Layer 2	Linear (256→128) → BatchNorm1d (128) → ReLU → Dropout (p=0.5)
Hidden Layer 3	Linear (128→64) → BatchNorm1d (64) → ReLU → Dropout (p=0.5)
Output Layer	Linear (64→1) – 1 giá trị logit cho phân loại nhị phân

Tổng số tham số (*parameters*) của mô hình: **64.641**.

2.2 Cấu hình huấn luyện

Table 5: Cấu hình huấn luyện MLP

Tham số	Giá trị
Hàm mất mát (<i>Loss function</i>)	BCEWithLogitsLoss, không dùng <code>pos_weight</code> (train đã cân bằng bằng SMOTE)
Thuật toán tối ưu (<i>Optimizer</i>)	Adam – <code>lr = 0.001</code> , <code>weight_decay = 0.001</code>
Scheduler	ReduceLROnPlateau – theo dõi <code>val_loss</code>
Epochs tối đa	100
Batch size	256
Early stopping	Patience = 15 epochs, theo dõi <code>val_loss</code>
Device	CPU

2.3 Các kỹ thuật được áp dụng

2.3.1 BatchNorm1d (chuẩn hóa theo lô – Batch Normalization)

Chuẩn hóa đầu ra của mỗi lớp tuyến tính trước khi đưa vào ReLU. Kỹ thuật này giúp ổn định gradient trong quá trình lan truyền ngược (*backpropagation*), giảm sự phụ thuộc vào giá trị *learning rate* ban đầu và tăng tốc độ hội tụ.

2.3.2 Dropout(p=0.5)

Trong mỗi bước huấn luyện, 50% nơ-ron bị tắt ngẫu nhiên, buộc mô hình không phụ thuộc vào bất kỳ nơ-ron đơn lẻ nào. Đây là kỹ thuật *regularization* (điều chuẩn) chính để giảm *overfitting*. Dropout chỉ hoạt động khi huấn luyện; khi đánh giá (*eval mode*), toàn bộ nơ-ron được kích hoạt trở lại.

2.3.3 Early Stopping (dừng huấn luyện sớm, patience = 15)

Sau mỗi epoch, `val_loss` được ghi nhận. Nếu `val_loss` không cải thiện trong 15 epoch liên tiếp, quá trình huấn luyện dừng lại và trọng số tốt nhất (*best checkpoint*) được phục hồi để đánh giá. Điều này tránh lãng phí tài nguyên và ngăn mô hình học thuộc dữ liệu train quá mức.

2.3.4 ReduceLROnPlateau

Kết hợp với Early Stopping, scheduler này tự động giảm *learning rate* theo hệ số nhỏ dần khi `val_loss` không cải thiện, giúp mô hình hội tụ mịn hơn ở giai đoạn cuối thay vì dao động xung quanh điểm tối ưu.

2.4 Diễn biến quá trình huấn luyện

Mô hình dừng sớm tại **epoch 24** (early stopping kích hoạt), với **val_loss** tốt nhất đạt được là **0.3546** tại **epoch 9**.

Table 6: Các mốc epoch tiêu biểu trong quá trình huấn luyện (*) = cải thiện **val_loss** tốt nhất

Epoch	train_loss	train_acc	val_loss	val_acc	
1	0.4202	0.8057	0.3885	0.8041	*
2	0.3824	0.8263	0.3818	0.8094	*
3	0.3772	0.8297	0.3554	0.8262	*
5	0.3714	0.8312	0.3918	0.8125	
9	0.3687	0.8322	0.3546	0.8222	*
10	0.3674	0.8333	0.3655	0.8156	
15	0.3638	0.8347	0.3745	0.8110	
20	0.3523	0.8403	0.3983	0.7933	
24	<i>Early stopping kích hoạt – phục hồi trọng số tốt nhất từ epoch 9</i>				

Nhận xét về diễn biến quá trình huấn luyện:

- **train_loss** giảm đều từ epoch 1 đến epoch 20, trong khi **val_loss** bắt đầu tăng trở lại sau epoch 9 – đây là dấu hiệu của *overfitting* nhẹ từ epoch 10 trở đi.
- Khoảng cách giữa **train_acc** và **val_acc** không quá lớn (≈ 0.84 so với ≈ 0.82), cho thấy mô hình vẫn tổng quát hóa ở mức chấp nhận được.
- Early stopping kích hoạt đúng lúc tại epoch 24, ngăn mô hình tiếp tục học theo hướng *overfitting* và phục hồi trọng số tốt nhất từ epoch 9.

2.5 Kết quả trên tập Test

Table 7: Kết quả MLP [256, 128, 64] – Test Set

Nhãn	Precision	Recall	F1-score	Support
$\leq 50K$	0.95	0.78	0.85	6.803
$> 50K$	0.56	0.87	0.68	2.242

Accuracy: 0.7992 **Precision** ($> 50K$): 0.5616 **Recall** ($> 50K$): 0.8662

F1-Score ($> 50K$): 0.6814 **ROC-AUC:** 0.9074

2.6 Nhận xét

MLP đạt Recall cao nhất cho lớp $>50K$ (0.8662 so với 0.8443 của Logistic Regression), nghĩa là mô hình phát hiện được nhiều trường hợp thu nhập cao thực sự hơn. ROC-AUC = 0.9074 cũng nhỉnh hơn Logistic Regression (0.9038), cho thấy khả năng tổng quát hóa tốt hơn khi xét toàn bộ ngưỡng quyết định. Tuy nhiên, sự đánh đổi là Precision thấp hơn một chút (0.5616 so với 0.5719), tức là MLP có xu hướng dự đoán dương nhiều hơn, kéo theo một số trường hợp *dương tính giả* (false positive).

3 So sánh tổng hợp: Logistic Regression và MLP

3.1 Bảng so sánh

Table 8: So sánh hiệu năng hai mô hình trên tập Test
(Precision, Recall, F1-Score tính trên lớp $>50K$)

Mô hình	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression (baseline)	0.8048	0.5719	0.8443	0.6819	0.9038
MLP [256, 128, 64]	0.7992	0.5616	0.8662	0.6814	0.9074

(Giá trị in đậm: tốt hơn giữa hai mô hình ở từng chỉ số.)

3.2 Phân tích từng chỉ số

3.2.1 Accuracy (độ chính xác tổng thể)

Logistic Regression cao hơn (0.8048 so với 0.7992). Tuy nhiên, với bài toán mất cân bằng lớp, accuracy một mình không đủ để đánh giá chất lượng mô hình một cách toàn diện, vì mô hình có thể đạt accuracy cao đơn thuần bằng cách thiên về dự đoán lớp đa số.

3.2.2 Precision (độ chính xác dự đoán dương – lớp $>50K$)

Logistic Regression cao hơn (0.5719 so với 0.5616). Khi Logistic Regression dự đoán một người có thu nhập $>50K$, xác suất đúng cao hơn MLP một chút, tức là Logistic Regression tạo ra ít *dương tính giả* (false positive) hơn trên mỗi dự đoán dương.

3.2.3 Recall (độ phủ lớp dương – lớp $>50K$)

MLP vượt trội rõ ràng (0.8662 so với 0.8443, hơn ≈ 2.2 điểm phần trăm). MLP phát hiện được nhiều trường hợp thu nhập cao thực sự hơn, tuy nhiên phải đánh đổi một phần Precision. Đây là đặc điểm quan trọng khi ưu tiên không bỏ sót lớp thiểu số.

3.2.4 F1-Score (trung bình điều hòa của Precision và Recall – lớp >50K)

Hai mô hình gần như tương đương (0.6819 so với 0.6814, chênh lệch chỉ 0.0005). Sự đánh đổi Precision–Recall ngược chiều nhau giữa hai mô hình dẫn đến F1-Score hầu như cân bằng, cho thấy không có mô hình nào vượt trội rõ ràng khi xét tổng thể cả hai yếu tố này.

3.2.5 ROC-AUC (diện tích dưới đường cong ROC – đo khả năng phân biệt hai lớp)

MLP cao hơn (0.9074 so với 0.9038). Đây là chỉ số tổng quát nhất, không phụ thuộc vào ngưỡng quyết định cụ thể. MLP có khả năng xếp hạng và phân biệt hai lớp thu nhập tốt hơn Logistic Regression ở mức tổng thể, thể hiện rằng mô hình học sâu nắm bắt được các mối quan hệ phi tuyến trong dữ liệu tốt hơn.

3.3 Kết luận

Hai mô hình có hiệu năng tổng thể rất gần nhau trên tập test. MLP thể hiện ưu thế ở **Recall** và **ROC-AUC**, cho thấy mạng nơ-ron có khả năng nắm bắt tốt hơn các mẫu thuộc lớp thiểu số >50K. Ngược lại, Logistic Regression vẫn là một baseline mạnh: đơn giản hơn, hội tụ nhanh hơn và cho Precision cùng Accuracy cao hơn.

Xét theo mục tiêu bài toán:

- Nếu ưu tiên **phát hiện đúng nhiều người có thu nhập >50K** (giảm thiểu *false negative* – bỏ sót lớp dương): MLP là lựa chọn phù hợp hơn nhờ Recall và ROC-AUC cao hơn.
- Nếu ưu tiên **độ chính xác của các dự đoán dương** và tốc độ triển khai thực tế: Logistic Regression là lựa chọn hợp lý hơn với chi phí tính toán thấp hơn đáng kể.

4 Đánh giá chung

4.1 Ưu điểm đã đạt được

- **Tính module hóa cao:** Kiến trúc MLP được thiết kế rõ ràng, đóng gói hoàn toàn trong `modules/deep_learning.py`, dễ tái sử dụng và điều chỉnh mà không ảnh hưởng đến các phần còn lại của pipeline.
- **Kiểm soát overfitting từ nhiều hướng:** Áp dụng đồng thời BatchNorm, Dropout và Early Stopping – ba kỹ thuật bổ trợ nhau, tác động lên các khía cạnh khác nhau của quá trình huấn luyện để giảm thiểu overfitting hiệu quả.
- **Quy trình đánh giá khách quan:** Sử dụng tập validation riêng để theo dõi Early Stopping, đảm bảo tập test hoàn toàn không tham gia vào bất kỳ quyết định nào trong quá trình huấn luyện hay chọn mô hình.

- **Đánh giá toàn diện:** So sánh đầy đủ với 5 chỉ số (Accuracy, Precision, Recall, F1-Score, ROC-AUC) và biểu đồ thanh trực quan, thay vì chỉ dựa vào một chỉ số duy nhất.
- **Không có data leakage:** Kết quả nhất quán giữa val và test ở cả hai mô hình xác nhận toàn bộ quy trình preprocessing (OHE, Scaler, SMOTE) được thực hiện đúng – fit trên train, transform trên val/test.

4.2 Hạn chế và điểm cần cải thiện

- **Không gian kiến trúc hẹp:** Chỉ thử nghiệm một kiến trúc MLP duy nhất [256, 128, 64]. Chưa so sánh với các biến thể khác như mạng nông hơn, sâu hơn, hoặc các cấu trúc có *skip connection* kiểu residual.
- **Chưa có hyperparameter tuning có hệ thống:** Các siêu tham số (learning rate, dropout rate, số lớp, số nơ-ron) được chọn thủ công dựa trên kinh nghiệm, chưa được tối ưu hóa một cách có phương pháp.
- **Cải thiện F1-Score chưa rõ ràng:** MLP chưa vượt trội so với Logistic Regression trên F1-Score (0.6814 so với 0.6819). Điều này có thể do bản chất dữ liệu dạng bảng (*tabular*) không phải thể mạnh điển hình của mạng nơ-ron sâu, hoặc do kiến trúc và tham số hiện tại chưa đạt mức tối ưu.
- **Khả năng giải thích hạn chế:** Mô hình MLP hoạt động như một *hộp đen* (black box), khó giải thích lý do đưa ra từng quyết định cụ thể, trong khi Logistic Regression cho phép phân tích trực tiếp thông qua hệ số hồi quy của từng đặc trưng.

5 Hướng phát triển tiếp theo

Dựa trên các hạn chế đã xác định ở phần trước, nhóm đề xuất các hướng cải thiện và mở rộng cụ thể cho giai đoạn hoàn thiện cuối kỳ.

5.1 Thử nghiệm thêm kiến trúc MLP

Kiến trúc [256, 128, 64] hiện tại chỉ là điểm khởi đầu. Nhóm dự kiến thử nghiệm thêm ít nhất hai biến thể để đánh giá ảnh hưởng của độ sâu và độ rộng mạng lên hiệu năng:

Table 9: Các kiến trúc MLP dự kiến thử nghiệm

Kiến trúc	Đặc điểm	Mục tiêu so sánh
[256, 128, 64] (hiện tại)	3 lớp ẩn, trung bình	Mốc tham chiếu
[128, 64]	2 lớp ẩn, nhỏ hơn	Kiểm tra xem mạng nông hơn có đủ khả năng biểu diễn không
[512, 256, 128, 64]	4 lớp ẩn, sâu hơn	Kiểm tra xem thêm lớp có giúp cải thiện ROC-AUC không

5.2 Hyperparameter Tuning có hệ thống (tinh chỉnh siêu tham số)

Thay vì chọn tham số thủ công, nhóm dự kiến áp dụng tìm kiếm tự động trên không gian tham số sau:

- **Learning rate:** {0.01; 0.001; 0.0001}
- **Dropout rate:** {0.3; 0.4; 0.5}
- **Weight decay** (*L2 regularization* – điều chuẩn L2): {0.0001; 0.001; 0.01}
- **Batch size:** {128; 256; 512}

Công cụ dự kiến sử dụng: **Optuna** (tìm kiếm siêu tham số tự động theo phương pháp *Bayesian optimization* – tối ưu hóa Bayes) hoặc **GridSearchCV** đơn giản nếu thời gian hạn chế. Tiêu chí chọn mô hình tốt nhất: ROC-AUC trên tập validation.

5.3 Xử lý mất cân bằng lớp – so sánh thêm phương pháp

Hiện tại nhóm chỉ sử dụng SMOTE để cân bằng lớp. Để đánh giá khách quan hơn, nhóm dự kiến thử thêm hai hướng tiếp cận:

- **Class weight trong loss function:** gán trọng số cao hơn cho lớp >50K trực tiếp trong `BCEWithLogitsLoss` thông qua tham số `pos_weight`, thay vì tăng số lượng mẫu bằng SMOTE. Cách này không làm thay đổi kích thước tập dữ liệu và tránh các mẫu tổng hợp có thể nhiễu.
- **Threshold tuning (điều chỉnh ngưỡng quyết định):** thay vì dùng ngưỡng mặc định 0.5, tìm ngưỡng tối ưu trên tập val để tối đa hóa F1-Score hoặc cân bằng Precision-Recall theo mục tiêu cụ thể của bài toán.

5.4 Hoàn thiện notebook và cấu trúc dự án

- Đảm bảo toàn bộ notebook `deep_learning.ipynb` chạy thành công với “Run All” trên Google Colab mà không cần can thiệp thủ công.

- Thêm đầy đủ *learning curves* (biểu đồ `train_loss` / `val_loss` và `train_acc` / `val_acc` theo từng epoch) để minh họa trực quan quá trình hội tụ.
- Lưu mô hình MLP đã huấn luyện (định dạng `.pt`) và các artifact tiền xử lý (`one_encoder`, `scaler`) vào thư mục `features/` để tái sử dụng mà không cần huấn luyện lại.
- Cập nhật README trên GitHub với hướng dẫn chạy đầy đủ, link notebook Colab và link báo cáo PDF cuối kỳ.

5.5 Báo cáo cuối kỳ

Báo cáo PDF cuối kỳ sẽ tổng hợp toàn bộ pipeline từ EDA đến Deep Learning, bao gồm các nội dung sau:

- So sánh toàn diện tất cả các mô hình đã thử nghiệm trong cả hai pipeline (truyền thống và Deep Learning): Decision Tree, Random Forest, SVM, Logistic Regression và MLP.
- Phân tích tầm quan trọng đặc trưng (*feature importance*) cho các mô hình hỗ trợ (Decision Tree, Random Forest) để giải thích kết quả và hiểu rõ hơn về ảnh hưởng của từng biến đầu vào.
- Bảng so sánh tổng hợp cuối cùng với đề xuất mô hình phù hợp nhất theo từng tiêu chí triển khai thực tế (độ chính xác, tốc độ suy luận, khả năng giải thích).